

Java Control Statements

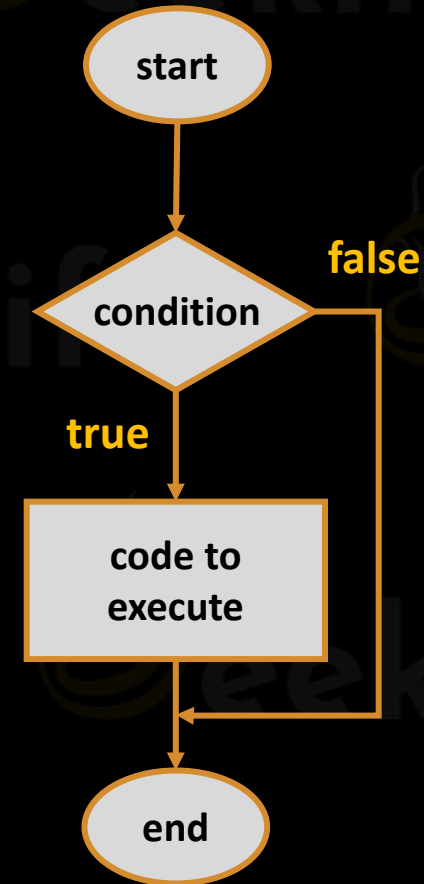


if-statement
if-else statement
if-else-if statement
nested-if statement
switch statement
switch expression
for-loop / for-each loop
while-loop
do-while loop
break / continue statements



if statement

```
if (condition) {  
    // code to be executed  
    // only if condition is true  
}
```



if statement

```
int var = 4;  
if (var % 2 == 0) {  
    System.out.println("I am Even");  
}  
System.out.println("Default Path");
```

Output:
I am Even
Default Path

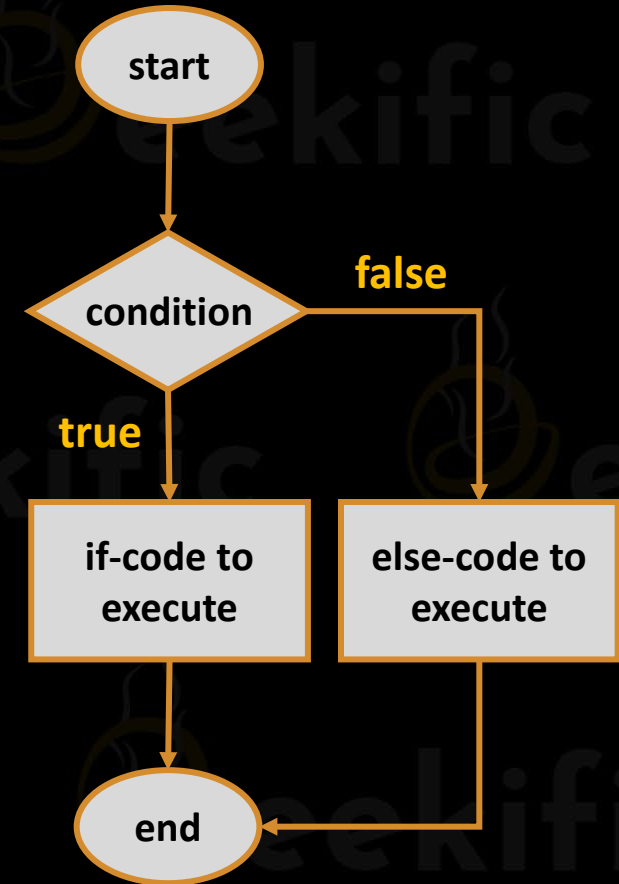
```
int var = 5;  
if (var % 2 == 0) {  
    System.out.println("I am Even");  
}  
System.out.println("Default Path");
```

Output:
Default Path



if-else statement

```
if (condition) {  
    // code to execute  
    // condition is true  
}  
else {  
    // code to execute  
    // condition is false  
}
```



ternary operator(?:)

```
String s = (condition) ? "condition is true" : "condition is false";
```



if-else statement

```
int var = 4;  
if (var % 2 == 0) {  
    var = var + 1;  
} else {  
    var = var + 2;  
}
```

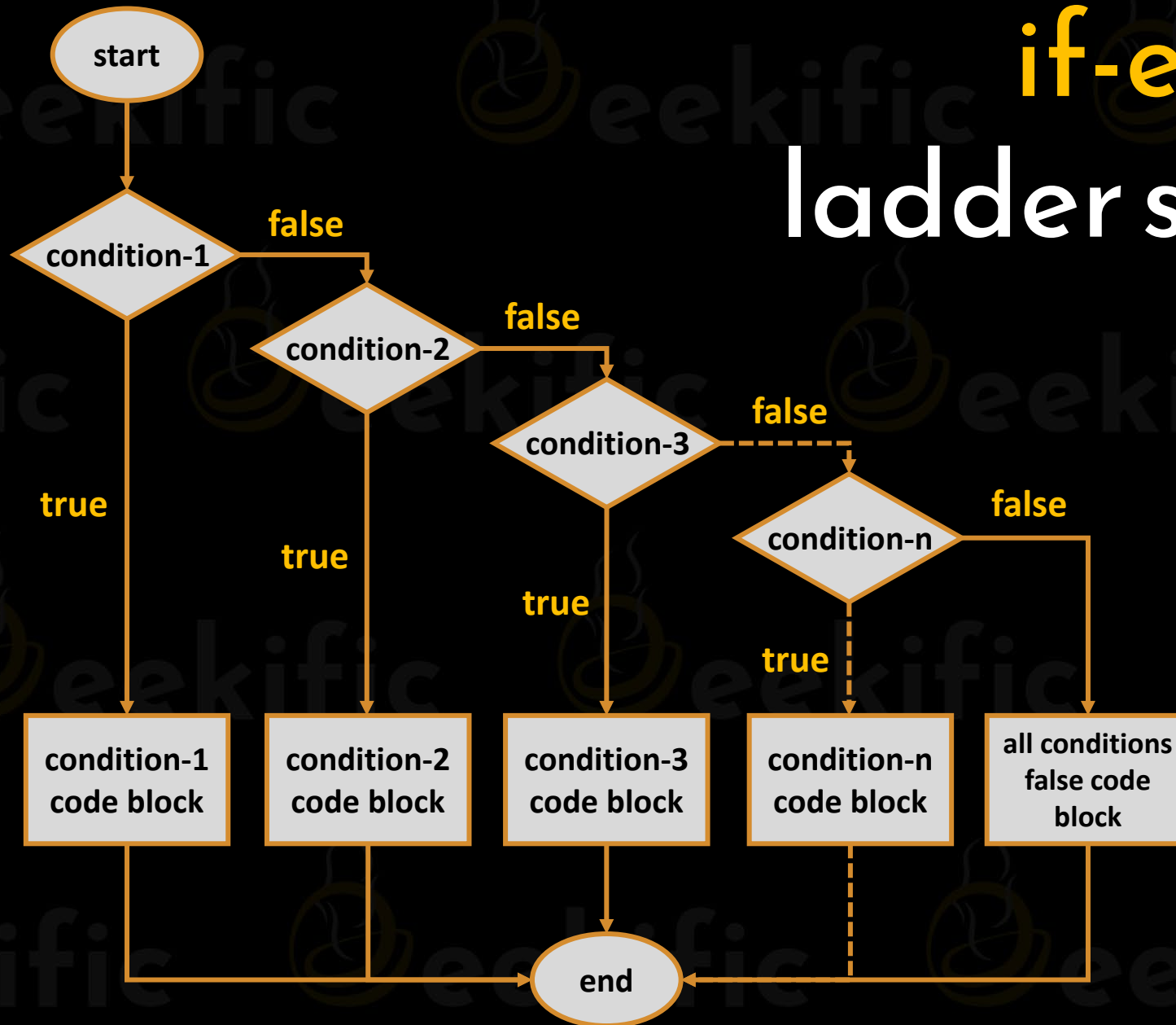
Output:
var = 5

ternary operator (?:)

```
int var = 4;  
var = (var % 2 == 0) ? var + 1 : var + 2;
```



if-else-if ladder statement



if-else-if ladder statement

```
if (condition-1) {  
    // code to execute if condition1 is true  
} else if (condition-2) {  
    // code to execute if condition1=false and condition2=true  
} else if (condition-3) {  
    // code to execute if cond1=cond2=false and condition3=true  
} ... {  
} else {  
    // code to execute if all conditions are false  
}
```



if-else-if ladder statement

```
int var = 7;  
if (var % 2 == 0) {  
    System.out.println("Multiple of 2");  
} else if (var % 5 == 0) {  
    System.out.println("Multiple of 5");  
} else if (var % 3 == 0) {  
    System.out.println("Multiple of 3");  
} else {  
    System.out.println("Geekific");  
}
```

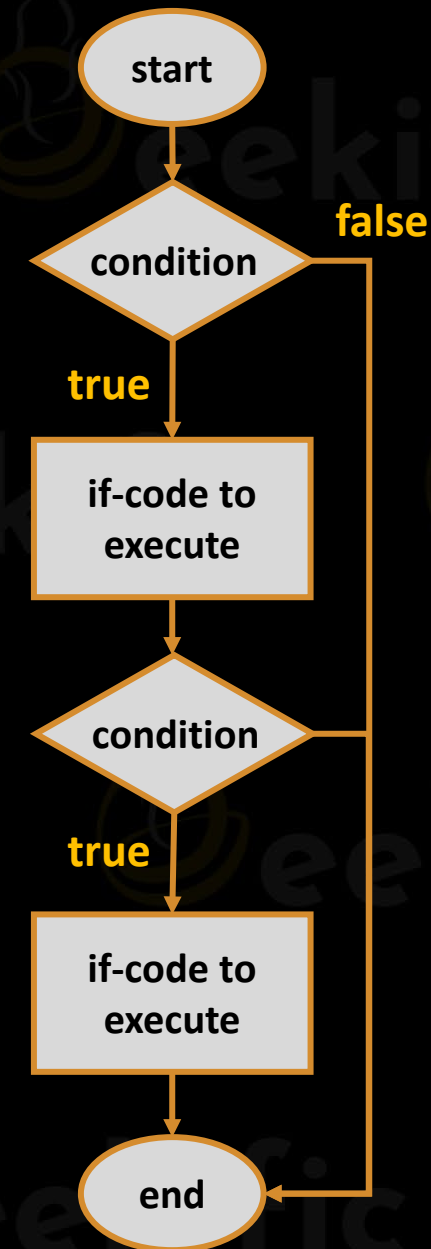
Output:
Geekific



nested-if statement

```
if (condition1 && condition2) {  
    // code to be executed  
}
```

```
if (condition1) {  
    // code to be executed  
    if (condition2) {  
        // code to be executed  
    }  
}
```



nested-if statement

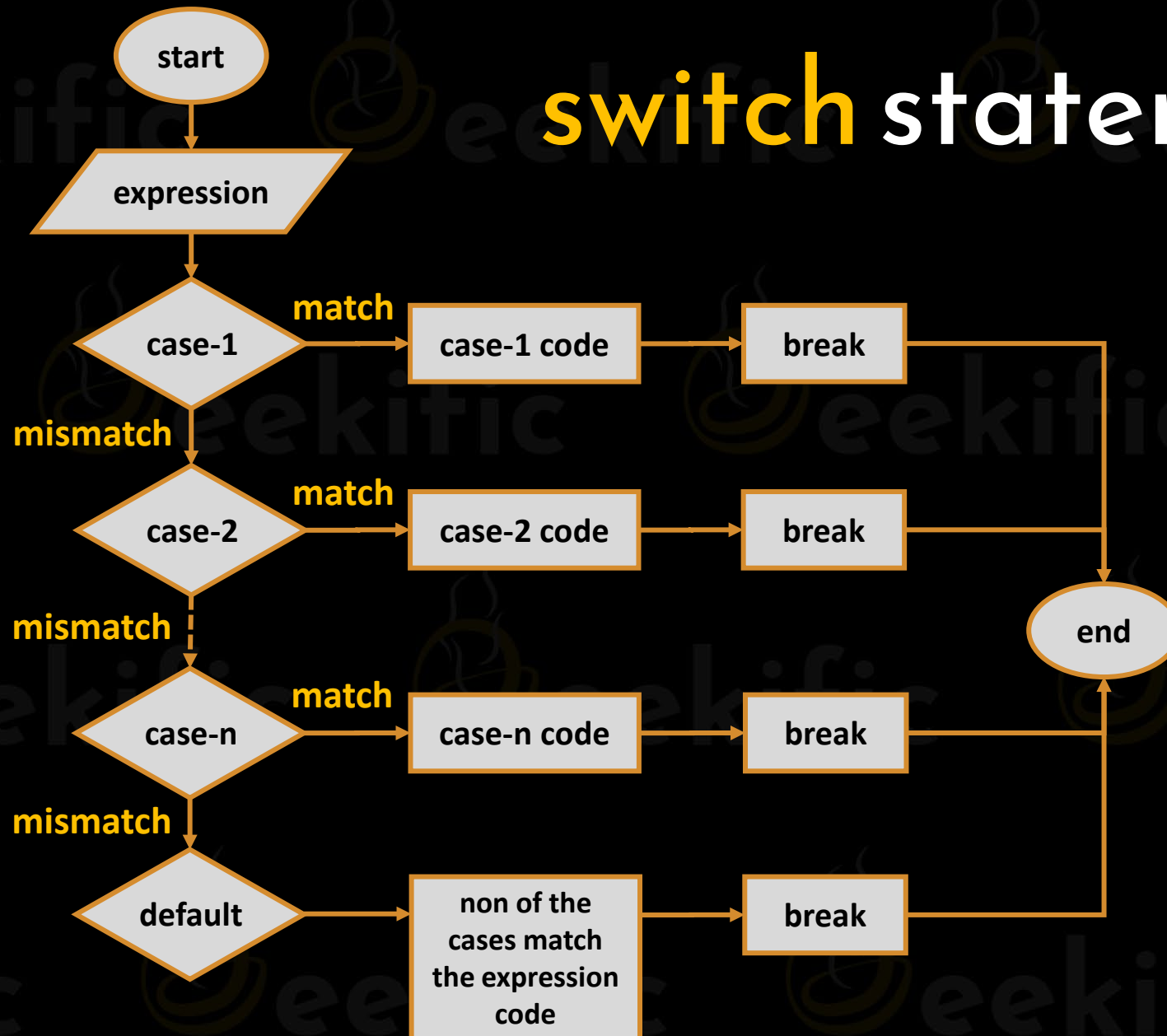
```
int var = 6;  
if (var % 2 == 0) {  
    System.out.println("I am a multiple of 2");  
    if (var % 3 == 0) {  
        System.out.println("I am also a multiple of 3");  
    }  
}
```

Output:

```
I am a multiple of 2  
I am also a multiple of 3
```



switch statement



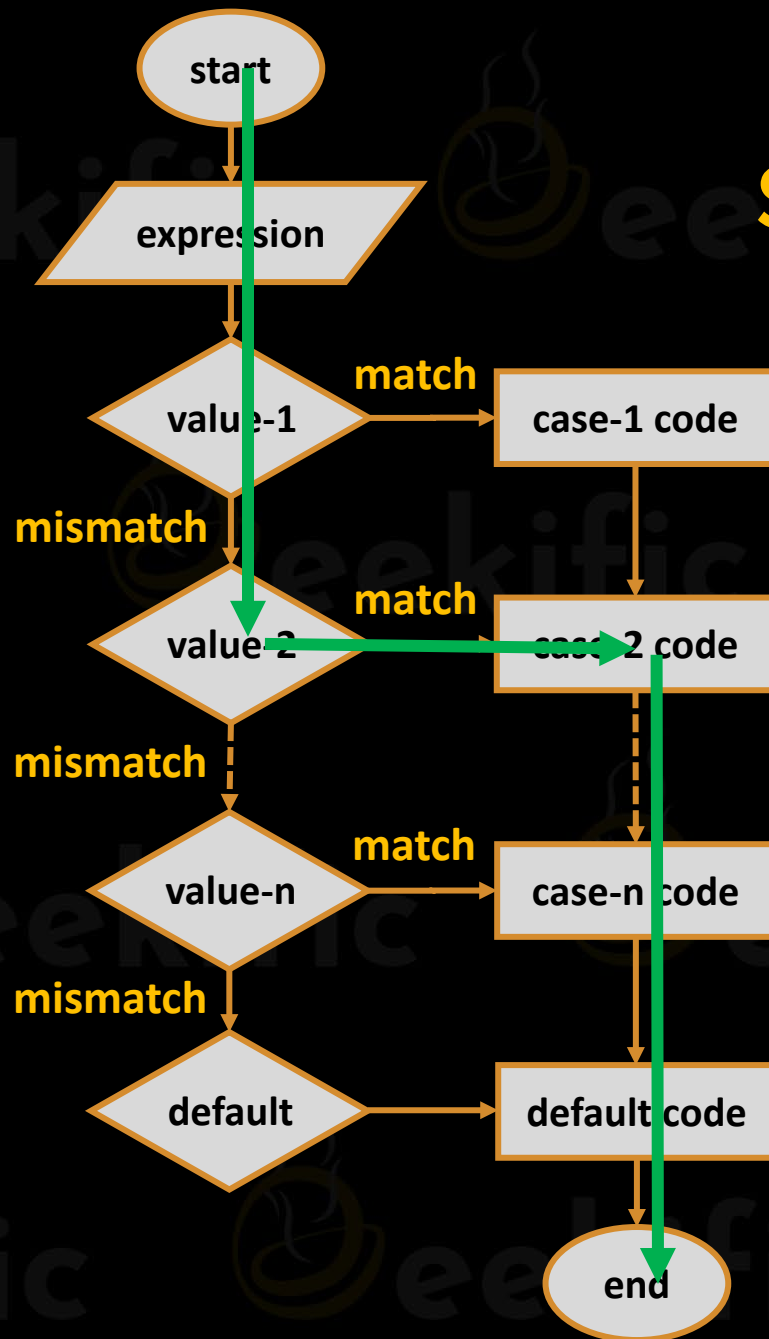
switch statement

```
switch(expression) {  
  
    case value-1:  
        // code to be executed if value1 equals the expression  
        break;  
  
    case value-2:  
        // code to execute if value-2 equals the expression  
        break;  
  
    .  
    .  
  
    default:  
        // code to execute if none of the values match the expression  
  
}
```



switch statement

expression
matches
case-2



if breaks are not
present, the Java
switch statement
will fall-through



Java Enum

```
public enum Day {  
  
    MONDAY,  
    TUESDAY,  
    WEDNESDAY,  
    THURSDAY,  
    FRIDAY,  
    SATURDAY,  
    SUNDAY  
  
}
```

think of an enum as a class which has a fixed set of constants that cannot be changed



enum & switch statements

```
switch(today) {  
  
    case MONDAY:  
        print("Today is Monday!"); break;  
  
    case TUESDAY:  
        print("Today is Tuesday!"); break;  
  
    case SATURDAY:  
  
    case SUNDAY:  
        print("Enjoy the Weekend!"); break;  
    .  
    .  
    .  
}
```

prior to Java 12



switch statements

```
switch(today) {  
    case MONDAY, TUESDAY, WEDNESDAY:  
  
    case THURSDAY, FRIDAY:  
        print("The week is not over yet!");  
        break;  
  
    case SATURDAY, SUNDAY:  
        print("Enjoy the weekend!");  
        break;  
}
```

multiple
values

post Java 12



switch expressions

```
int result = switch(today) {  
    case MONDAY, TUESDAY, WEDNESDAY -> 1;  
    case THURSDAY, FRIDAY -> 2;  
    case SATURDAY -> 3;  
    default -> 0;  
};
```

post Java 13



switch expressions

```
int result = switch(today) {  
    case MONDAY, TUESDAY, WEDNESDAY -> 1;  
    case THURSDAY, FRIDAY -> {  
        int length = today.toString().length();  
        yield length * length;  
    }  
    case SATURDAY -> 3;  
    default -> 0;  
};
```

post Java 13



switch expressions

```
switch (today) {  
    case MONDAY, TUESDAY -> return 3;  
}
```



```
int result = switch(today) {  
    case MONDAY, TUESDAY -> 1;  
};
```

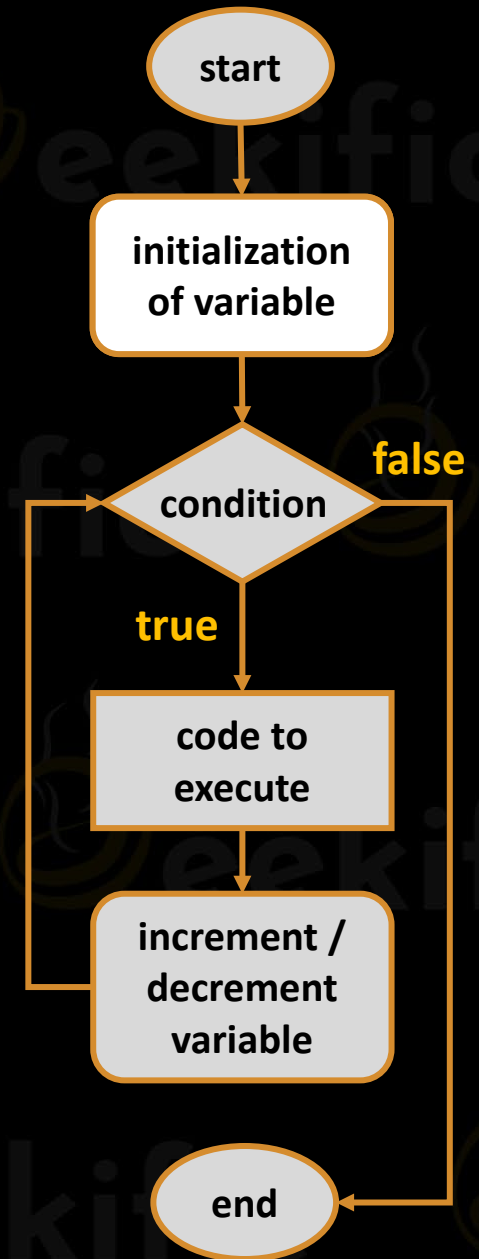


```
int result = switch(today) {  
    case MONDAY, TUESDAY -> 1;  
    default -> 0;  
};
```



for loop

```
for (initialize; condition; increment/decrement) {  
    // code to be executed  
}
```



for loop

```
for (int i = 0; i < 3; i++) {  
    System.out.println(i);  
}
```

Output:

```
0 // i = 0  
1 // i = 1  
2 // i = 2
```



for loop

```
for (int i = 1; i < 10; i*=2) {
```

```
for (int i = 1; i < 10; i = i - 2 * var) {
```

```
for (int var = 1; var < 10; var = getNewValue(var)) {
```

```
    System.out.println(var);
```

```
}
```



for loop

```
for (int i = 0, j = 1; i <= 5; i++, j++) {  
    System.out.println(i + j);  
}
```

Output:

1
3
5
7
9
11



nested for loops

```
for (initialization; condition; incr/decr) {  
    // we can also execute code here  
  
    for (initialization; condition; incr/decr) {  
        // code to be executed  
    }  
}
```



nested for loops

```
for (int outer = 0; outer < 3; outer++) {  
    for (int inner = 0; inner < 3; inner ++){  
        System.out.println("(" + outer + ", " + inner + ")");  
    }  
}
```

Output:

(0, 0)	(0, 1)	(0, 2)
(1, 0)	(1, 1)	(1, 2)
(2, 0)	(2, 1)	(2, 2)



for-each loop

```
for (Type variable : array) {  
    // code to be executed  
}
```

```
for (Type variable : list) {  
    // code to be executed  
}
```



for-each loop

```
List<String> list = Arrays.asList("for", "each", "with", "Geekific");  
for (String var : list) {  
    System.out.println(var);  
}
```

Output:

for
each
with
Geekific



labeled for loop

```
labelName:  
for (Type variable : array) {  
    // code to be executed  
}
```

```
label1:  
for (initialization; condition; incr/decr) {  
    // we can also execute code here  
    label2:  
    for (initialization; condition; incr/decr) {  
        // code to be executed  
    }  
}
```

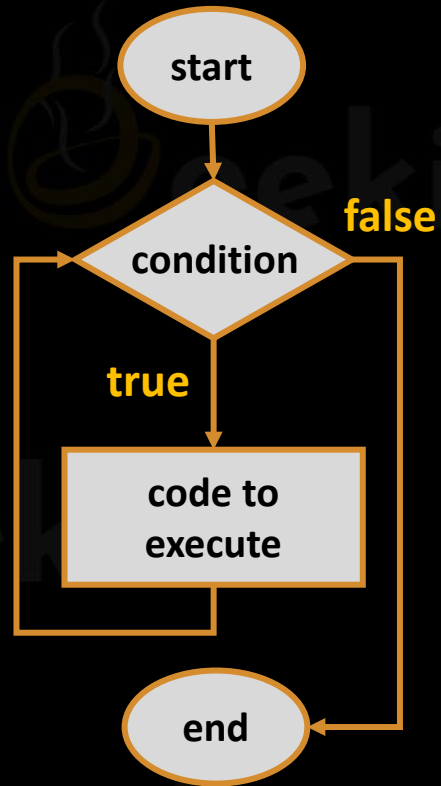


infinite **for** loop

```
for (;;) {  
    // code to be executed  
}
```



while loop



```
while (condition) {  
    // code to be executed  
}
```



while loop

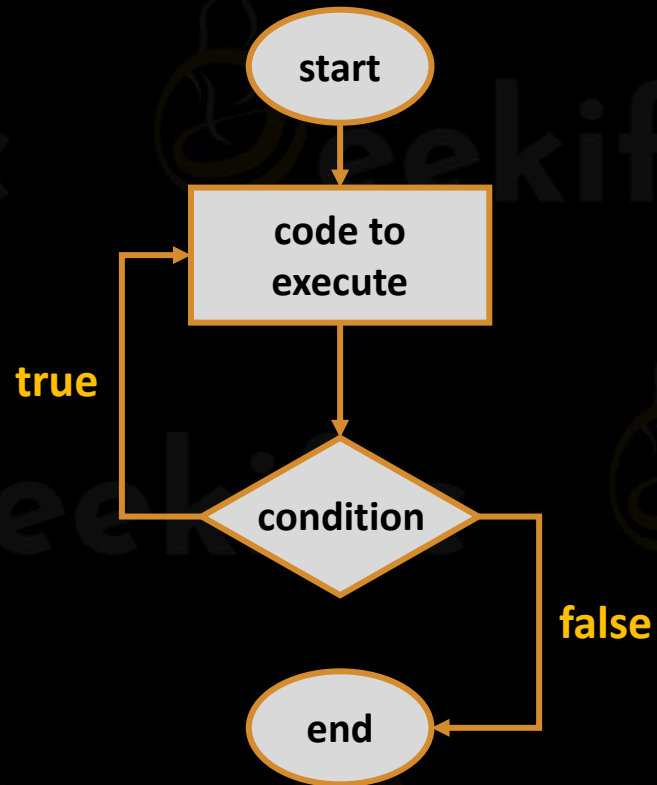
```
int var = 0;  
while (var < 3) {  
    System.out.println(var);  
    var++;  
}
```

Output:

0
1
2



do-while loop



```
do {  
    // code to be executed  
} while (condition);
```



do-while loop

```
int var = 0;  
do {  
    System.out.println(var);  
    var++;  
} while (var < 3);
```

Output:

0
1
2



while vs do-while

```
do {  
    // code will be executed once  
} while (false);
```

do-while loop: code will
execute at least once

exit-controlled
statement



while vs do-while

```
do {  
    // code will be executed once  
} while (false);
```

```
while (false) {  
    // code will never be executed  
}
```

while loop: code
executed only based on
the condition

entry-controlled
statement



while vs do-while

```
while (condition) {  
    // code will never be executed  
}
```

```
do {  
    // code will be executed once  
} while (condition);
```

a semicolon
is required at the end of
the do-while loop, but
none is needed in case
of the while loop



while vs do-while

```
while (true) {  
    // code will never be executed  
}
```

```
do {  
    // code will be executed once  
} while (true);
```

to create infinite loops, in both cases it is done by setting the condition to true



break statement

```
for (initialization; condition; incr/decr) {  
    if (anotherCondition) {  
        // will exit the for loop  
        break;  
    }  
}
```

```
while (condition) {  
    if (anotherCondition) {  
        // will exit the while loop  
        break;  
    }  
}
```

```
do {  
    if (anotherCondition) {  
        // will exit the do-while loop  
        break;  
    }  
} while (condition);
```



break statement

```
for (initialization; condition; incr/decr) {  
    for (initialization; condition; incr/decr) {  
        if (anotherCondition) {  
            // will exit the inner for loop  
            // the outer loop will keep going  
            break;  
        }  
    }  
}
```



break statement

```
label1:
for (initialization; condition; incr/decr) {
    for (initialization; condition; incr/decr) {
        if (anotherCondition) {
            // if we use labels we can break
            // out of any loop we want
            break label1;
        }
    }
}
```



break statement

```
geekific:
for (int outer = 0; outer < 3; outer++) {
    for (int inner = 0; inner < 3; inner++) {

        System.out.println("(" + outer + ", " + inner + ")");

        if (inner == 0 && outer == 0) {
            break geekific;
        }

    }
}
```

Output:

(0, 0)



continue statement

```
for (initialization; condition; incr/decr) {  
    if (anotherCondition) {  
        continue;  
    }  
    // if the continue statement is hit  
    // this code will NOT be executed  
    // for THIS increment  
}
```

```
while (condition) {  
    if (anotherCondition) {  
        continue;  
    }  
    // if anotherCondition is met  
    // this code will NOT be executed  
    // for THIS increment  
}
```

```
do {  
    if (anotherCondition) {  
        continue;  
    }  
    // if the continue statement is hit  
    // the rest of the loop will be SKIPPED  
    // for THIS increment  
} while (condition);
```



continue statement

```
for (initialization; condition; incr/decr) {  
    for (initialization; condition; incr/decr) {  
        if (anotherCondition) {  
            continue;  
        }  
        // code written here will NOT be executed  
        // if the anotherCondition is true  
    }  
}
```



continue statement

```
label1:
for (initialization; condition; incr/decr) {
    for (initialization; condition; incr/decr) {

        if (anotherCondition) {
            continue label1;
        }
        // if anotherCondition is true
        // the remaining iterations of this
        // inner for loop will be SKIPPED
        // and the outer for loop is incremented

    }
}
```



continue statement

```
geekific:
for (int outer = 0; outer < 3; outer++) {

    for (int inner = 0; inner < 3; inner++) {

        System.out.println("(" + outer + ", " + inner + ")");
        continue geekific;

    }

}
```

Output:

```
(0, 0)
(1, 0)
(2, 0)
```

